MODULE 8

GitHub Repository Link: https://github.com/devcnx/csd440

Source Code

Create Table — CREATE TABLE IF NOT EXISTS

BrittaneyCreateTable.php

```
<?php
/**
 * Module 8.2 Programming Assignment — Create Table
 * CSD440 Server-Side Scripting
 *
 * Creates the bperrymorgan_ollama_models table inside the course-supplied
 * baseball_01 database using MySQLi. The table catalogs locally available
 * Ollama language models — chosen as a topic because it ties into ongoing
 * AI-enablement work and naturally exercises multiple data types.
 *
 * Schema (8 columns, 5 distinct data types):
 *   - model_id       INT         AUTO_INCREMENT PRIMARY KEY
 *   - model_name     VARCHAR(100) UNIQUE
 *   - parameter_count DECIMAL(6,2) — model size in billions of parameters
 *   - quantization   VARCHAR(20)  — e.g., Q4_K_M, F16
 *   - size_gb        DECIMAL(6,2) — disk footprint in gigabytes
 *   - release_date   DATE
 *   - is_installed   TINYINT(1)   — boolean flag, 1 = pulled locally
 *   - use_case       VARCHAR(255)
 *
 * Intended run order: Create → Populate → Query. Drop is available for
 * cleanup between test runs.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-03
 * @php     >= 8.0
 */

// ── Configuration ──────────────────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
```

```
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Issues the CREATE TABLE statement against the active connection.
 *
 * Uses CREATE TABLE IF NOT EXISTS so the script is safely idempotent.
 * Re-running it does not error out when the table already exists.
 *
 * @param  mysqli      $mysqli Active MySQLi connection.
 * @param  string|null &$error Populated with the failure reason on error.
 * @return bool                True if the table now exists, false on error.
 */
function createOllamaModelsTable(mysqli $mysqli, ?string &$error = null): bool
{
    $sql = "CREATE TABLE IF NOT EXISTS " . DB_TABLE . " (
        model_id       INT         NOT NULL AUTO_INCREMENT,
        model_name     VARCHAR(100) NOT NULL UNIQUE,
        parameter_count DECIMAL(6,2) NOT NULL,
        quantization    VARCHAR(20)  NOT NULL,
        size_gb        DECIMAL(6,2) NOT NULL,
        release_date    DATE        NOT NULL,
        is_installed    TINYINT(1)   NOT NULL DEFAULT 0,
        use_case       VARCHAR(255) NOT NULL,
        PRIMARY KEY (model_id)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4";

    if (!$mysqli->query($sql)) {
        $error = $mysqli->error;
        return false;
    }
    return true;
}

/**
 * Reads the live table schema via DESCRIBE for display confirmation.
 *
 * Returns the column descriptors as an array of associative rows. An
 * empty array indicates the DESCRIBE query failed or the table is missing.
 *
 * @param  mysqli $mysqli Active MySQLi connection.
 * @return array          List of column descriptors (Field, Type, Null, …).
 */
```

```
function describeTable(mysqli $mysqli): array
{
   $rows = [];
   $result = $mysqli->query("DESCRIBE " . DB_TABLE);
   if ($result instanceof mysqli_result) {
      while ($row = $result->fetch_assoc()) {
         $rows[] = $row;
      }
      $result->free();
   }
   return $rows;
}

// ── Execute ──────────────────────────────────────────────────────────────────────────────
$connectError = null;
$createError  = null;
$schemaRows   = [];
$tableCreated = false;

$mysqli = connectToDatabase($connectError);
if ($mysqli !== null) {
   $tableCreated = createOllamaModelsTable($mysqli, $createError);
   if ($tableCreated) {
      $schemaRows = describeTable($mysqli);
   }
   $mysqli->close();
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 8.2 Programming Assignment — Create Table';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Create Table — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
   <link rel="stylesheet" href="/modules/shared.css">
</head>

<body>
   <h1>Create Table</h1>

   <div class="header-info">
      <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
      <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
      <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
   </div>
```

```
<?php if ($connectError !== null): ?>
  <div class="error-summary">
    <h2>Connection Failed</h2>
    <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code> database.</p>
    <ul class="error-list">
      <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
    </ul>
    <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
  </div>

<?php elseif (!$tableCreated): ?>
  <div class="error-summary">
    <h2>Create Failed</h2>
    <p class="error-message">The <code><?php echo DB_TABLE; ?></code> table could not be created.</p>
    <ul class="error-list">
      <li><?php echo htmlspecialchars($createError ?? 'Unknown error.', ENT_QUOTES, 'UTF-8'); ?></li>
    </ul>
  </div>

<?php else: ?>
  <div class="confirmation">
    <h2>Table Ready</h2>
    <p class="success-message">
      The <code><?php echo DB_TABLE; ?></code> table is in place
      (created if missing) inside the <code><?php echo DB_NAME; ?></code> database.
    </p>
    <div class="query-note">
      <strong>SQL executed:</strong>
      <code>CREATE TABLE IF NOT EXISTS <?php echo DB_TABLE; ?> ( … 8 columns … )</code>
    </div>
    <p>Next step: populate the table by running
      <code><a href="BrittaneyPopulateTable.php">BrittaneyPopulateTable.php</a></code>.</p>
  </div>

  <h2>Confirmed Schema</h2>
  <table>
    <caption>DESCRIBE <?php echo DB_TABLE; ?></caption>
    <thead>
      <tr>
        <th>Field</th>
        <th>Type</th>
        <th>Null</th>
        <th>Key</th>
        <th>Default</th>
        <th>Extra</th>
      </tr>
    </thead>
    <tbody>
<?php foreach ($schemaRows as $row): ?>
      <tr>
        <td><code><?php echo htmlspecialchars($row['Field'], ENT_QUOTES, 'UTF-8'); ?></code></td>
        <td><?php echo htmlspecialchars($row['Type'], ENT_QUOTES, 'UTF-8'); ?></td>
```

```
            <td><?php echo htmlspecialchars($row['Null'], ENT_QUOTES, 'UTF-8'); ?></td>
            <td><?php echo htmlspecialchars($row['Key'], ENT_QUOTES, 'UTF-8'); ?></td>
            <td><?php echo htmlspecialchars((string)($row['Default'] ?? ''), ENT_QUOTES, 'UTF-8'); ?></td>
            <td><?php echo htmlspecialchars($row['Extra'], ENT_QUOTES, 'UTF-8'); ?></td>
        </tr>
<?php endforeach; ?>
      </tbody>
   </table>
<?php endif; ?>

   <div class="form-actions">
      <a href="BrittaneyPopulateTable.php">Populate Table</a>
      <a href="BrittaneyQueryTable.php" class="btn-secondary">Query Table</a>
      <a href="BrittaneyDropTable.php" class="btn-secondary">Drop Table</a>
   </div>
</body>

</html>
```

Populate Table — INSERT with Prepared Statements

BrittaneyPopulateTable.php

```
<?php
/**
 * Module 8.2 Programming Assignment — Populate Table
 * CSD440 Server-Side Scripting
 *
 * Inserts seed data into the bperrymorgan_ollama_models table inside the
 * course-supplied baseball_01 database. Uses a parameterized prepared
 * statement (mysqli::prepare + bind_param) so the load step also serves
 * as a working demonstration of the safe-DB-access pattern from M7.1.
 *
 * The seed dataset describes eight Ollama models that span a realistic
 * range of parameter sizes, quantizations, release dates, and on-disk
 * footprints — chosen so the M9 query work has interesting data to filter
 * and aggregate over.
 *
 * Idempotency: the INSERT uses ON DUPLICATE KEY UPDATE keyed on
 * model_name so re-running this script refreshes existing rows rather
 * than throwing duplicate-key errors. The summary distinguishes inserts
 * from updates so the result is unambiguous.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-03
 * @php     >= 8.0
 */

// ── Configuration ──────────────────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';
```

```
// Seed rows, ordered by release date.
// Each row: [model_name, parameter_count_billions, quantization,
//           size_gb, release_date, is_installed, use_case]
const SEED_ROWS = [
    ['llava:13b',              13.00, 'Q4_K_M', 8.00, '2024-01-30', 0, 'Vision-language model for image understanding'],
    ['nomic-embed-text:latest',   0.14, 'F16',    0.27, '2024-02-01', 1, 'Text embeddings for RAG pipelines'],
    ['phi3:mini',               3.80, 'Q4_K_M', 2.30, '2024-04-23', 1, 'Lightweight reasoning on limited hardware'],
    ['gemma2:9b',               9.00, 'Q4_K_M', 5.44, '2024-06-27', 1, 'Open Google model, balanced performance'],
    ['mistral-nemo:12b',        12.00, 'Q4_K_M', 7.07, '2024-07-18', 1, 'Multilingual chat with long context window'],
    ['llama3.1:8b',             8.00, 'Q4_K_M', 4.92, '2024-07-23', 1, 'General-purpose chat and coding assistance'],
    ['llama3.1:70b',            70.00, 'Q4_K_M',39.50, '2024-07-23', 0, 'High-quality reasoning, slower local inference'],
    ['qwen2.5-coder:7b',         7.00, 'Q4_K_M', 4.36, '2024-09-19', 1, 'Code generation and refactoring'],
];

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null        Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Inserts the seed dataset into the table using a prepared statement.
 *
 * Loops the SEED_ROWS constant, binding each row's values into the same
 * prepared INSERT to avoid SQL injection and re-parsing overhead. The
 * ON DUPLICATE KEY UPDATE clause keys on model_name (the UNIQUE column),
 * so re-running the script refreshes rows rather than failing.
 *
 * affected_rows semantics under ON DUPLICATE KEY UPDATE:
 *   1 = new row inserted
 *   2 = existing row updated
 *   0 = existing row matched but no values changed (counted as update)
 *
 * @param  mysqli      $mysqli Active MySQLi connection.
 * @param  string|null &$error Populated with the failure reason on error.
 * @return array               Counts: ['inserted' => int, 'updated' => int,
```

```
 *                                  'total' => int, 'failed' => int].
 */
function populateOllamaModelsTable(mysqli $mysqli, ?string &$error = null): array
{
    $sql = "INSERT INTO " . DB_TABLE . "
        (model_name, parameter_count, quantization, size_gb, release_date, is_installed, use_case)
        VALUES (?, ?, ?, ?, ?, ?, ?)
        ON DUPLICATE KEY UPDATE
            parameter_count = VALUES(parameter_count),
            quantization    = VALUES(quantization),
            size_gb         = VALUES(size_gb),
            release_date    = VALUES(release_date),
            is_installed    = VALUES(is_installed),
            use_case        = VALUES(use_case)";

    $stmt = $mysqli->prepare($sql);
    if (!$stmt) {
        $error = $mysqli->error;
        return ['inserted' => 0, 'updated' => 0, 'total' => count(SEED_ROWS), 'failed' => count(SEED_ROWS)];
    }

    $inserted = 0;
    $updated  = 0;
    $failed   = 0;

    foreach (SEED_ROWS as $row) {
        // Type string: s = string, d = decimal/double, i = integer
        // Order matches the VALUES (?, ?, ?, ?, ?, ?, ?) above.
        $name       = $row[0];
        $params     = $row[1];
        $quant      = $row[2];
        $sizeGb     = $row[3];
        $releaseDate = $row[4];
        $installed   = $row[5];
        $useCase     = $row[6];

        $stmt->bind_param('sdsdsis', $name, $params, $quant, $sizeGb, $releaseDate, $installed, $useCase);

        if (!$stmt->execute()) {
            $failed++;
            continue;
        }

        // affected_rows: 1 = insert, 2 = update via ON DUPLICATE KEY,
        // 0 = matched but unchanged. Treat 0 and 2 both as "already there."
        if ($stmt->affected_rows === 1) {
            $inserted++;
        } else {
            $updated++;
        }
    }
    $stmt->close();
```

```
    return [
        'inserted' => $inserted,
        'updated'  => $updated,
        'total'    => count(SEED_ROWS),
        'failed'   => $failed,
    ];
}

/**
 * Reads every row currently in the table for post-load verification.
 *
 * @param  mysqli $mysqli Active MySQLi connection.
 * @return array          List of row associative arrays, ordered by model_id.
 */
function fetchAllRows(mysqli $mysqli): array
{
    $rows = [];
    $result = $mysqli->query("SELECT * FROM " . DB_TABLE . " ORDER BY model_id");
    if ($result instanceof mysqli_result) {
        while ($row = $result->fetch_assoc()) {
            $rows[] = $row;
        }
        $result->free();
    }
    return $rows;
}

// ── Execute ──────────────────────────────────────────────────────────────────
$connectError  = null;
$populateError = null;
$counts        = ['inserted' => 0, 'updated' => 0, 'total' => 0, 'failed' => 0];
$rows          = [];
$populated     = false;

$mysqli = connectToDatabase($connectError);
if ($mysqli !== null) {
    $counts    = populateOllamaModelsTable($mysqli, $populateError);
    $populated = $populateError === null && $counts['failed'] === 0;
    if ($populateError === null) {
        $rows = fetchAllRows($mysqli);
    }
    $mysqli->close();
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 8.2 Programming Assignment — Populate Table';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">
```

```
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Populate Table — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
   <link rel="stylesheet" href="/modules/shared.css">
</head>

<body>
   <h1>Populate Table</h1>

   <div class="header-info">
      <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
      <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
      <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
   </div>

<?php if ($connectError !== null): ?>
   <div class="error-summary">
      <h2>Connection Failed</h2>
      <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code> database.</p>
      <ul class="error-list">
         <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
      </ul>
      <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
   </div>

<?php elseif ($populateError !== null): ?>
   <div class="error-summary">
      <h2>Populate Failed</h2>
      <p class="error-message">The prepared INSERT could not be initialized.</p>
      <ul class="error-list">
         <li><?php echo htmlspecialchars($populateError, ENT_QUOTES, 'UTF-8'); ?></li>
      </ul>
      <p>If the error mentions a missing table, run
         <code><a href="BrittaneyCreateTable.php">BrittaneyCreateTable.php</a></code> first.</p>
   </div>

<?php else: ?>
   <div class="confirmation">
      <h2>Seed Load Complete</h2>
      <p class="success-message">
         Processed <?php echo (int)$counts['total']; ?> rows against
         <code><?php echo DB_TABLE; ?></code>:
         <strong><?php echo (int)$counts['inserted']; ?></strong> inserted,
         <strong><?php echo (int)$counts['updated']; ?></strong> updated.
<?php if ($counts['failed'] > 0): ?>
         <span class="error"><?php echo (int)$counts['failed']; ?> failed.</span>
<?php endif; ?>
      </p>
      <div class="query-note">
         <strong>SQL pattern:</strong>
         <code>INSERT INTO <?php echo DB_TABLE; ?> ( … ) VALUES (?, ?, ?, ?, ?, ?, ?) ON DUPLICATE KEY UPDATE
```

```
…</code>
        — bound seven times per row using <code>mysqli::prepare</code> + <code>bind_param</code>.
      </div>
      <p>Next step: verify with
        <code><a href="BrittaneyQueryTable.php">BrittaneyQueryTable.php</a></code>.</p>
    </div>

    <h2>Table Contents After Load</h2>
    <table>
      <caption>SELECT * FROM <?php echo DB_TABLE; ?> ORDER BY model_id</caption>
      <thead>
        <tr>
          <th>ID</th>
          <th>Model</th>
          <th>Params (B)</th>
          <th>Quant</th>
          <th>Size (GB)</th>
          <th>Released</th>
          <th>Installed</th>
          <th>Use Case</th>
        </tr>
      </thead>
      <tbody>
<?php if (empty($rows)): ?>
        <tr><td colspan="8" class="empty">No rows present.</td></tr>
<?php else: foreach ($rows as $row): ?>
        <tr>
          <td><?php echo (int)$row['model_id']; ?></td>
          <td><code><?php echo htmlspecialchars($row['model_name'], ENT_QUOTES, 'UTF-8'); ?></code></td>
          <td><?php echo htmlspecialchars($row['parameter_count'], ENT_QUOTES, 'UTF-8'); ?></td>
          <td><?php echo htmlspecialchars($row['quantization'], ENT_QUOTES, 'UTF-8'); ?></td>
          <td><?php echo htmlspecialchars($row['size_gb'], ENT_QUOTES, 'UTF-8'); ?></td>
          <td><?php echo htmlspecialchars($row['release_date'], ENT_QUOTES, 'UTF-8'); ?></td>
          <td>
<?php if ((int)$row['is_installed'] === 1): ?>
            <span class="flag-true">Yes</span>
<?php else: ?>
            <span class="flag-false">No</span>
<?php endif; ?>
          </td>
          <td><?php echo htmlspecialchars($row['use_case'], ENT_QUOTES, 'UTF-8'); ?></td>
        </tr>
<?php endforeach; endif; ?>
      </tbody>
    </table>
<?php endif; ?>

    <div class="form-actions">
      <a href="BrittaneyQueryTable.php">Query Table</a>
      <a href="BrittaneyCreateTable.php" class="btn-secondary">Create Table</a>
      <a href="BrittaneyDropTable.php" class="btn-secondary">Drop Table</a>
    </div>
</body>
```

```
</html>
```

Query Table — Four SELECT Queries

BrittaneyQueryTable.php

```
<?php
/**
 * Module 8.2 Programming Assignment — Query Table
 * CSD440 Server-Side Scripting
 *
 * Runs four SELECT queries against the bperrymorgan_ollama_models table
 * inside the course-supplied baseball_01 database to demonstrate the
 * table is populated and behaves as expected. Each query exercises a
 * different SQL feature so the demo also covers grading rubric breadth:
 *
 *   1. Full-table read with explicit column ordering.
 *   2. Filtered read (WHERE + ORDER BY) — locally installed models only.
 *   3. Aggregate read (COUNT, SUM, AVG, ROUND) — fleet summary.
 *   4. Grouped read (GROUP BY quantization) — counts per quant scheme.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-03
 * @php     >= 8.0
 */

// ── Configuration ──────────────────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
```

```
}

/**
 * Runs an arbitrary SELECT and returns the result rows.
 *
 * Wraps the raw mysqli::query result so the caller works only with plain
 * arrays. Errors are surfaced via the $error reference instead of being
 * thrown so the calling page can render multiple queries independently
 * (one failure should not abort the rest of the page).
 *
 * @param  mysqli      $mysqli Active MySQLi connection.
 * @param  string      $sql    The SQL SELECT statement to execute.
 * @param  string|null &$error Populated with the failure reason on error.
 * @return array               Result rows; empty array on error or no rows.
 */
function runSelect(mysqli $mysqli, string $sql, ?string &$error = null): array
{
    $rows = [];
    $result = $mysqli->query($sql);
    if (!($result instanceof mysqli_result)) {
        $error = $mysqli->error;
        return [];
    }
    while ($row = $result->fetch_assoc()) {
        $rows[] = $row;
    }
    $result->free();
    return $rows;
}

// ── Execute ──────────────────────────────────────────────────────────────────
$connectError = null;
$mysqli       = connectToDatabase($connectError);

// Each query gets its own error slot so a single failure does not blank
// the rest of the page.
$queries = [
    'all' => [
        'title' => 'All Models (newest first)',
        'sql'   => "SELECT model_id, model_name, parameter_count, quantization, size_gb, release_date, is_installed,
use_case
                    FROM " . DB_TABLE . "
                    ORDER BY release_date DESC, model_name",
        'rows'  => [],
        'error' => null,
    ],
    'installed' => [
        'title' => 'Locally Installed Models, Largest First',
        'sql'   => "SELECT model_name, parameter_count, size_gb, use_case
                    FROM " . DB_TABLE . "
                    WHERE is_installed = 1
                    ORDER BY parameter_count DESC",
        'rows'  => [],
```

```
        'error' => null,
    ],
    'summary' => [
        'title' => 'Fleet Summary (aggregates)',
        'sql'   => "SELECT
                    COUNT(*)                        AS total_models,
                    SUM(is_installed)               AS installed_count,
                    ROUND(SUM(size_gb), 2)          AS total_size_gb,
                    ROUND(AVG(parameter_count), 2)  AS avg_param_billions,
                    MAX(parameter_count)            AS largest_param_count,
                    MIN(release_date)               AS oldest_release,
                    MAX(release_date)               AS newest_release
                FROM " . DB_TABLE,
        'rows'  => [],
        'error' => null,
    ],
    'by_quant' => [
        'title' => 'Models Grouped by Quantization',
        'sql'   => "SELECT
                    quantization,
                    COUNT(*)                        AS model_count,
                    ROUND(AVG(size_gb), 2)          AS avg_size_gb
                FROM " . DB_TABLE . "
                GROUP BY quantization
                ORDER BY model_count DESC, quantization",
        'rows'  => [],
        'error' => null,
    ],
];

if ($mysqli !== null) {
    foreach ($queries as $key => $query) {
        $err = null;
        $queries[$key]['rows']  = runSelect($mysqli, $query['sql'], $err);
        $queries[$key]['error'] = $err;
    }
    $mysqli->close();
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 8.2 Programming Assignment — Query Table';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');

/**
 * Echoes a value or an em-dash when null/empty for cleaner table cells.
 *
 * @param  mixed $value Raw cell value from a result row.
 * @return string       HTML-safe display string.
 */
function cell(mixed $value): string
{
```

```
    if ($value === null || $value === '') {
        return '<span class="empty">—</span>';
    }
    return htmlspecialchars((string)$value, ENT_QUOTES, 'UTF-8');
}
?>
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Query Table — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
    <link rel="stylesheet" href="/modules/shared.css">
</head>

<body>
    <h1>Query Table</h1>

    <div class="header-info">
        <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
        <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
        <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
    </div>

<?php if ($connectError !== null): ?>
    <div class="error-summary">
        <h2>Connection Failed</h2>
        <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code> database.</p>
        <ul class="error-list">
            <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
        </ul>
        <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
    </div>

<?php else: ?>

    <h2>1. <?php echo htmlspecialchars($queries['all']['title'], ENT_QUOTES, 'UTF-8'); ?></h2>
    <div class="query-note">
        <strong>SQL:</strong>
        <code>SELECT * FROM <?php echo DB_TABLE; ?> ORDER BY release_date DESC, model_name</code>
    </div>
<?php if ($queries['all']['error'] !== null): ?>
    <p class="error-message">Query failed: <?php echo htmlspecialchars($queries['all']['error'], ENT_QUOTES, 'UTF-
8'); ?></p>
<?php else: ?>
    <table>
        <caption>All <?php echo count($queries['all']['rows']); ?> models</caption>
        <thead>
            <tr>
                <th>ID</th>
                <th>Model</th>
```

```
            <th>Params (B)</th>
            <th>Quant</th>
            <th>Size (GB)</th>
            <th>Released</th>
            <th>Installed</th>
            <th>Use Case</th>
          </tr>
        </thead>
        <tbody>
<?php if (empty($queries['all']['rows'])): ?>
          <tr><td colspan="8" class="empty">No rows. Run BrittaneyPopulateTable.php first.</td></tr>
<?php else: foreach ($queries['all']['rows'] as $row): ?>
          <tr>
            <td><?php echo (int)$row['model_id']; ?></td>
            <td><code><?php echo cell($row['model_name']); ?></code></td>
            <td><?php echo cell($row['parameter_count']); ?></td>
            <td><?php echo cell($row['quantization']); ?></td>
            <td><?php echo cell($row['size_gb']); ?></td>
            <td><?php echo cell($row['release_date']); ?></td>
            <td>
<?php if ((int)$row['is_installed'] === 1): ?>
                <span class="flag-true">Yes</span>
<?php else: ?>
                <span class="flag-false">No</span>
<?php endif; ?>
            </td>
            <td><?php echo cell($row['use_case']); ?></td>
          </tr>
<?php endforeach; endif; ?>
        </tbody>
      </table>
<?php endif; ?>

      <h2>2. <?php echo htmlspecialchars($queries['installed']['title'], ENT_QUOTES, 'UTF-8'); ?></h2>
      <div class="query-note">
        <strong>SQL:</strong>
        <code>SELECT model_name, parameter_count, size_gb, use_case FROM <?php echo DB_TABLE; ?> WHERE
is_installed = 1 ORDER BY parameter_count DESC</code>
      </div>
<?php if ($queries['installed']['error'] !== null): ?>
      <p class="error-message">Query failed: <?php echo htmlspecialchars($queries['installed']['error'], ENT_QUOTES,
'UTF-8'); ?></p>
<?php else: ?>
      <table>
        <caption><?php echo count($queries['installed']['rows']); ?> installed models</caption>
        <thead>
          <tr>
            <th>Model</th>
            <th>Params (B)</th>
            <th>Size (GB)</th>
            <th>Use Case</th>
          </tr>
        </thead>
```

```
      <tbody>
<?php if (empty($queries['installed']['rows'])): ?>
         <tr><td colspan="4" class="empty">No installed models found.</td></tr>
<?php else: foreach ($queries['installed']['rows'] as $row): ?>
         <tr>
            <td><code><?php echo cell($row['model_name']); ?></code></td>
            <td><?php echo cell($row['parameter_count']); ?></td>
            <td><?php echo cell($row['size_gb']); ?></td>
            <td><?php echo cell($row['use_case']); ?></td>
         </tr>
<?php endforeach; endif; ?>
      </tbody>
   </table>
<?php endif; ?>

   <h2>3. <?php echo htmlspecialchars($queries['summary']['title'], ENT_QUOTES, 'UTF-8'); ?></h2>
   <div class="query-note">
      <strong>SQL:</strong>
      <code>SELECT COUNT(*), SUM(is_installed), ROUND(SUM(size_gb),2), ROUND(AVG(parameter_count),2),
MAX(parameter_count), MIN(release_date), MAX(release_date) FROM <?php echo DB_TABLE; ?></code>
   </div>
<?php if ($queries['summary']['error'] !== null): ?>
   <p class="error-message">Query failed: <?php echo htmlspecialchars($queries['summary']['error'],
ENT_QUOTES, 'UTF-8'); ?></p>
<?php elseif (empty($queries['summary']['rows'])): ?>
   <p class="empty">No rows in table to aggregate.</p>
<?php else: $summary = $queries['summary']['rows'][0]; ?>
   <table class="data-display">
      <caption>Aggregate Statistics</caption>
      <tbody>
         <tr><td>Total models cataloged</td><td><?php echo cell($summary['total_models']); ?></td></tr>
         <tr><td>Installed locally</td><td><?php echo cell($summary['installed_count']); ?></td></tr>
         <tr><td>Combined disk footprint</td><td><?php echo cell($summary['total_size_gb']); ?> GB</td></tr>
         <tr><td>Average parameter count</td><td><?php echo cell($summary['avg_param_billions']); ?>
B</td></tr>
         <tr><td>Largest model</td><td><?php echo cell($summary['largest_param_count']); ?> B
parameters</td></tr>
         <tr><td>Oldest release date</td><td><?php echo cell($summary['oldest_release']); ?></td></tr>
         <tr><td>Newest release date</td><td><?php echo cell($summary['newest_release']); ?></td></tr>
      </tbody>
   </table>
<?php endif; ?>

   <h2>4. <?php echo htmlspecialchars($queries['by_quant']['title'], ENT_QUOTES, 'UTF-8'); ?></h2>
   <div class="query-note">
      <strong>SQL:</strong>
      <code>SELECT quantization, COUNT(*), ROUND(AVG(size_gb),2) FROM <?php echo DB_TABLE; ?> GROUP BY
quantization ORDER BY model_count DESC</code>
   </div>
<?php if ($queries['by_quant']['error'] !== null): ?>
   <p class="error-message">Query failed: <?php echo htmlspecialchars($queries['by_quant']['error'],
ENT_QUOTES, 'UTF-8'); ?></p>
<?php else: ?>
```

```
  <table>
    <caption>Counts per quantization scheme</caption>
    <thead>
      <tr>
        <th>Quantization</th>
        <th>Model Count</th>
        <th>Avg Size (GB)</th>
      </tr>
    </thead>
    <tbody>
<?php if (empty($queries['by_quant']['rows'])): ?>
      <tr><td colspan="3" class="empty">No rows to group.</td></tr>
<?php else: foreach ($queries['by_quant']['rows'] as $row): ?>
      <tr>
        <td><code><?php echo cell($row['quantization']); ?></code></td>
        <td><?php echo cell($row['model_count']); ?></td>
        <td><?php echo cell($row['avg_size_gb']); ?></td>
      </tr>
<?php endforeach; endif; ?>
    </tbody>
  </table>
<?php endif; ?>

<?php endif; /* connection ok */ ?>

  <div class="form-actions">
    <a href="BrittaneyCreateTable.php">Create Table</a>
    <a href="BrittaneyPopulateTable.php" class="btn-secondary">Populate Table</a>
    <a href="BrittaneyDropTable.php" class="btn-secondary">Drop Table</a>
  </div>
</body>

</html>
```

Drop Table — DROP TABLE IF EXISTS

BrittaneyDropTable.php

```
<?php
/**
 * Module 8.2 Programming Assignment — Drop Table
 * CSD440 Server-Side Scripting
 *
 * Drops the bperrymorgan_ollama_models table from the course-supplied
 * baseball_01 database using MySQLi. Used between test runs to reset the
 * environment to a clean state. Uses DROP TABLE IF EXISTS so the script
 * is safe to run even when the table has already been removed.
 *
 * Reports whether the table existed before the DROP so the user can tell
 * the difference between a successful cleanup and an idempotent no-op.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-03
 * @php     >= 8.0
```

```
 */

// ── Configuration ──────────────────────────────────────────────────────────
const DB_HOST  = 'localhost';
const DB_USER  = 'student1';
const DB_PASS  = 'pass';
const DB_NAME  = 'baseball_01';
const DB_TABLE = 'bperrymorgan_ollama_models';

/**
 * Establishes a MySQLi connection to the baseball_01 database.
 *
 * Disables MySQLi's default exception reporting so connection failure is
 * surfaced through the returned $error reference instead of an uncaught
 * exception, letting the caller render a clean HTML error page.
 *
 * @param  string|null &$error Populated with the failure reason on error.
 * @return mysqli|null         Live connection on success, null on failure.
 */
function connectToDatabase(?string &$error = null): ?mysqli
{
    mysqli_report(MYSQLI_REPORT_OFF);
    $mysqli = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($mysqli->connect_error) {
        $error = $mysqli->connect_error;
        return null;
    }
    $mysqli->set_charset('utf8mb4');
    return $mysqli;
}

/**
 * Checks whether a given table currently exists in the active database.
 *
 * Used before the DROP so the success message can distinguish "actually
 * removed something" from "no-op, table was already gone."
 *
 * @param  mysqli $mysqli Active MySQLi connection.
 * @param  string $table  Unqualified table name to check.
 * @return bool           True if the table exists, false otherwise.
 */
function tableExists(mysqli $mysqli, string $table): bool
{
    $escaped = $mysqli->real_escape_string($table);
    $result = $mysqli->query("SHOW TABLES LIKE '" . $escaped . "'");
    if (!($result instanceof mysqli_result)) {
        return false;
    }
    $exists = $result->num_rows > 0;
    $result->free();
    return $exists;
}
```

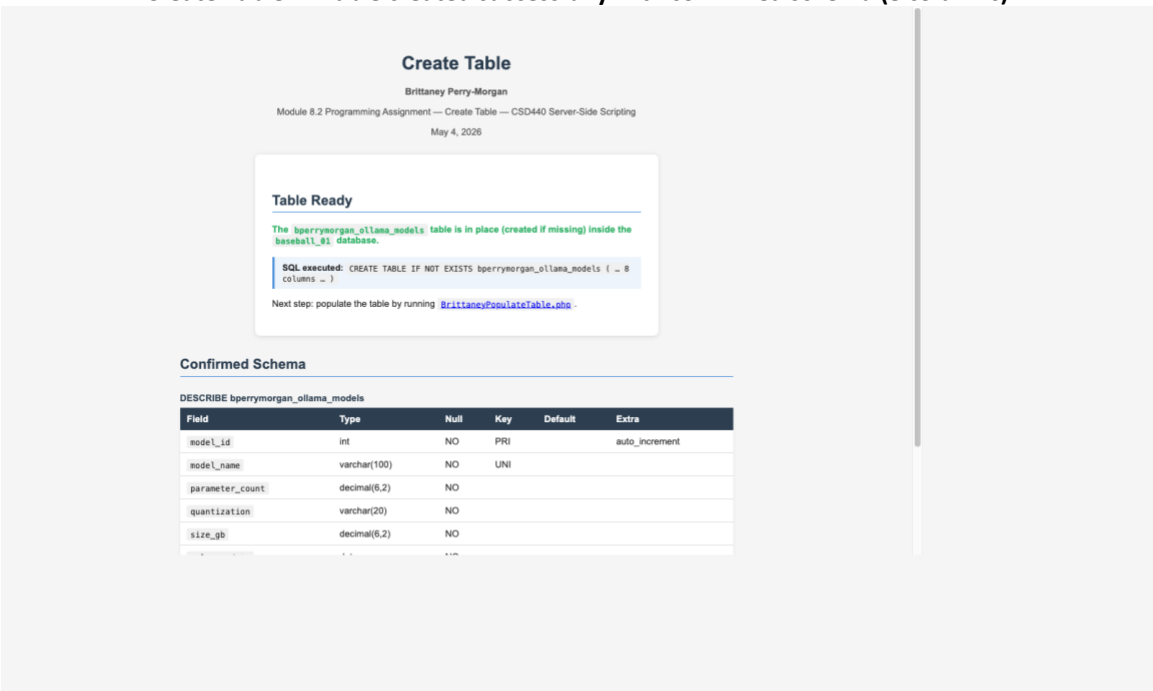
```
/**
 * Issues the DROP TABLE statement against the active connection.
 *
 * Uses DROP TABLE IF EXISTS so the script is safely idempotent — running
 * it on an already-dropped table does not error out.
 *
 * @param  mysqli      $mysqli Active MySQLi connection.
 * @param  string|null &$error Populated with the failure reason on error.
 * @return bool                True on success, false on error.
 */
function dropOllamaModelsTable(mysqli $mysqli, ?string &$error = null): bool
{
    if (!$mysqli->query("DROP TABLE IF EXISTS " . DB_TABLE)) {
        $error = $mysqli->error;
        return false;
    }
    return true;
}

// ── Execute ──────────────────────────────────────────────────────────────────
$connectError  = null;
$dropError     = null;
$existedBefore = false;
$dropped       = false;

$mysqli = connectToDatabase($connectError);
if ($mysqli !== null) {
    $existedBefore = tableExists($mysqli, DB_TABLE);
    $dropped       = dropOllamaModelsTable($mysqli, $dropError);
    $mysqli->close();
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 8.2 Programming Assignment — Drop Table';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Drop Table — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
    <link rel="stylesheet" href="/modules/shared.css">
</head>

<body>
    <h1>Drop Table</h1>

    <div class="header-info">
        <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8'); ?></strong></p>
```

```
      <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> — <?php echo
htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
      <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
   </div>

<?php if ($connectError !== null): ?>
   <div class="error-summary">
      <h2>Connection Failed</h2>
      <p class="error-message">Could not connect to the <code><?php echo DB_NAME; ?></code> database.</p>
      <ul class="error-list">
         <li><?php echo htmlspecialchars($connectError, ENT_QUOTES, 'UTF-8'); ?></li>
      </ul>
      <p>Verify MySQL is running and the <code><?php echo DB_USER; ?></code> credentials are valid.</p>
   </div>

<?php elseif (!$dropped): ?>
   <div class="error-summary">
      <h2>Drop Failed</h2>
      <p class="error-message">The <code><?php echo DB_TABLE; ?></code> table could not be dropped.</p>
      <ul class="error-list">
         <li><?php echo htmlspecialchars($dropError ?? 'Unknown error.', ENT_QUOTES, 'UTF-8'); ?></li>
      </ul>
   </div>

<?php else: ?>
   <div class="confirmation">
      <h2><?php echo $existedBefore ? 'Table Dropped' : 'Nothing To Drop'; ?></h2>

<?php if ($existedBefore): ?>
      <p class="success-message">
         The <code><?php echo DB_TABLE; ?></code> table was found and removed
         from the <code><?php echo DB_NAME; ?></code> database.
      </p>
<?php else: ?>
      <p class="success-message">
         The <code><?php echo DB_TABLE; ?></code> table did not exist, so the
         DROP statement completed as a no-op. The database is in a clean state.
      </p>
<?php endif; ?>

      <div class="query-note">
         <strong>SQL executed:</strong>
         <code>DROP TABLE IF EXISTS <?php echo DB_TABLE; ?></code>
      </div>

      <p>Next step: rebuild the table by running
         <code><a href="BrittaneyCreateTable.php">BrittaneyCreateTable.php</a></code>.</p>
   </div>
<?php endif; ?>

   <div class="form-actions">
      <a href="BrittaneyCreateTable.php">Create Table</a>
      <a href="BrittaneyPopulateTable.php" class="btn-secondary">Populate Table</a>
```

```
        <a href="BrittaneyQueryTable.php" class="btn-secondary">Query Table</a>
    </div>
</body>

</html>
```

Deliverables

Create Table — Table created successfully with confirmed schema (8 columns)



Create Table

Brittaney Perry-Morgan

Module 8.2 Programming Assignment — Create Table — CSD440 Server-Side Scripting

May 4, 2026

Table Ready

The `bperrymorgan_ollama_models` table is in place (created if missing) inside the `baseball_01` database.

SQL executed: `CREATE TABLE IF NOT EXISTS bperrymorgan_ollama_models ( … 8 columns … )`

Next step: populate the table by running `BrittaneyPopulateTable.php`.

Confirmed Schema

DESCRIBE bperrymorgan_ollama_models

Field	Type	Null	Key	Default	Extra
model_id	int	NO	PRI		auto_increment
model_name	varchar(100)	NO	UNI		
parameter_count	decimal(6,2)	NO			
quantization	varchar(20)	NO			
size_gb	decimal(6,2)	NO			

Create Table — DESCRIBE output showing full column definitions

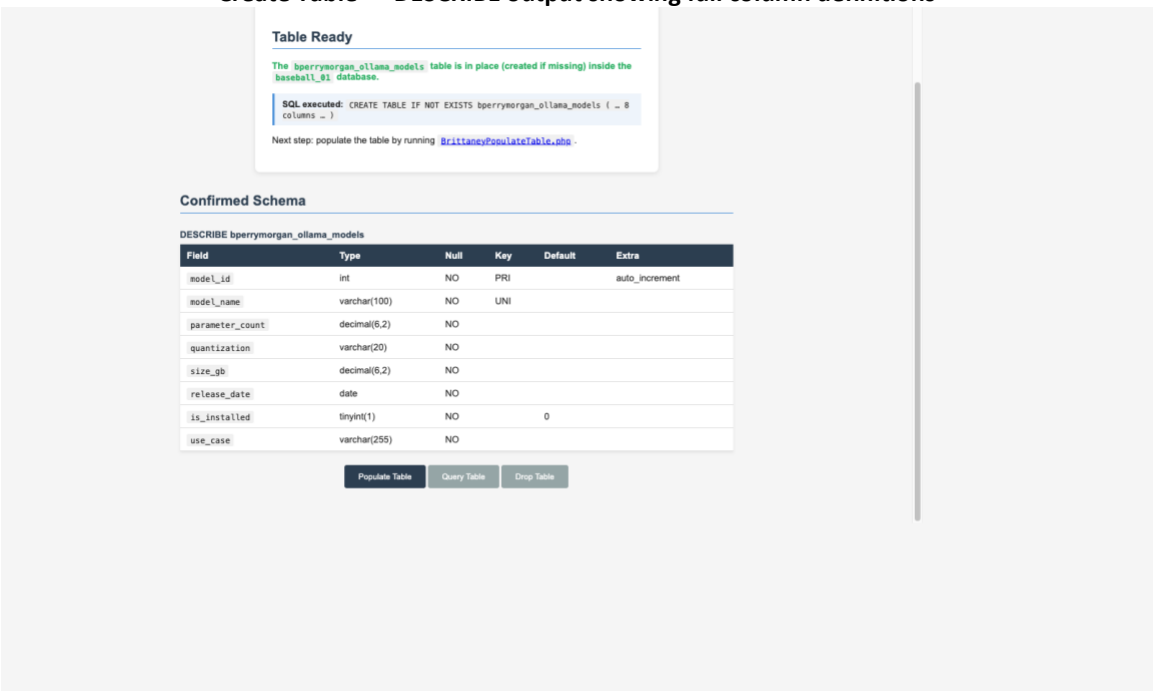


Table Ready

The `bperrymorgan_ollama_models` table is in place (created if missing) inside the `baseball_01` database.

SQL executed: `CREATE TABLE IF NOT EXISTS bperrymorgan_ollama_models ( … 8 columns … )`

Next step: populate the table by running `BrittaneyPopulateTable.php`.

Confirmed Schema

DESCRIBE bperrymorgan_ollama_models

Field	Type	Null	Key	Default	Extra
model_id	int	NO	PRI		auto_increment
model_name	varchar(100)	NO	UNI		
parameter_count	decimal(6,2)	NO			
quantization	varchar(20)	NO			
size_gb	decimal(6,2)	NO			
release_date	date	NO			
is_installed	tinyint(1)	NO		0	
use_case	varchar(255)	NO			

Populate Table | Query Table | Drop Table

Populate Table — First run: 8 rows inserted, 0 updated

Populate Table

Brittaney Perry-Morgan

Module 8.2 Programming Assignment — Populate Table — CSD440 Server-Side Scripting

May 4, 2026

Seed Load Complete

Processed 8 rows against `bperrymorgan_ollama_models` : 8 inserted, 0 updated.

SQL pattern: `INSERT INTO bperrymorgan_ollama_models ( … ) VALUES (?, ?, ?, ?, ?, ?, ?) ON DUPLICATE KEY UPDATE …` — bound seven times per row using `mysqli::prepare` + `bind_param` .

Next step: verify with `BrittaneyQueryTable.php` .

Table Contents After Load

SELECT * FROM bperrymorgan_ollama_models ORDER BY model_id

ID	Model	Params (B)	Quant	Size (GB)	Released	Installed	Use Case
1	`llava:13b`	13.00	Q4_K_M	8.00	2024-01-30	No	Vision-language model for image understanding
2	`nomic-embed-text:latest`	0.14	F16	0.27	2024-02-01	Yes	Text embeddings for RAG pipelines
3	`phi3:mini`	3.80	Q4_K_M	2.30	2024-04-23	Yes	Lightweight reasoning on limited hardware
					2024-06-		Open Google model, balanced

Populate Table — Table contents after initial load (8 rows)

Next step: verify with `BrittaneyQueryTable.php` .

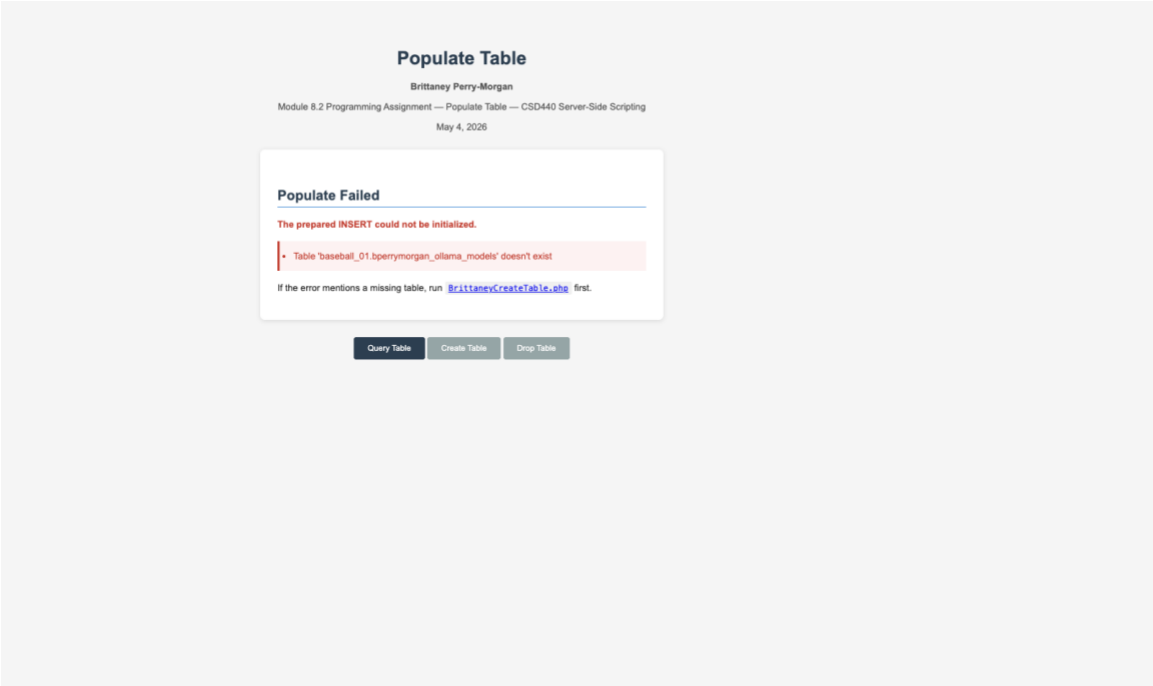
Table Contents After Load

SELECT * FROM bperrymorgan_ollama_models ORDER BY model_id

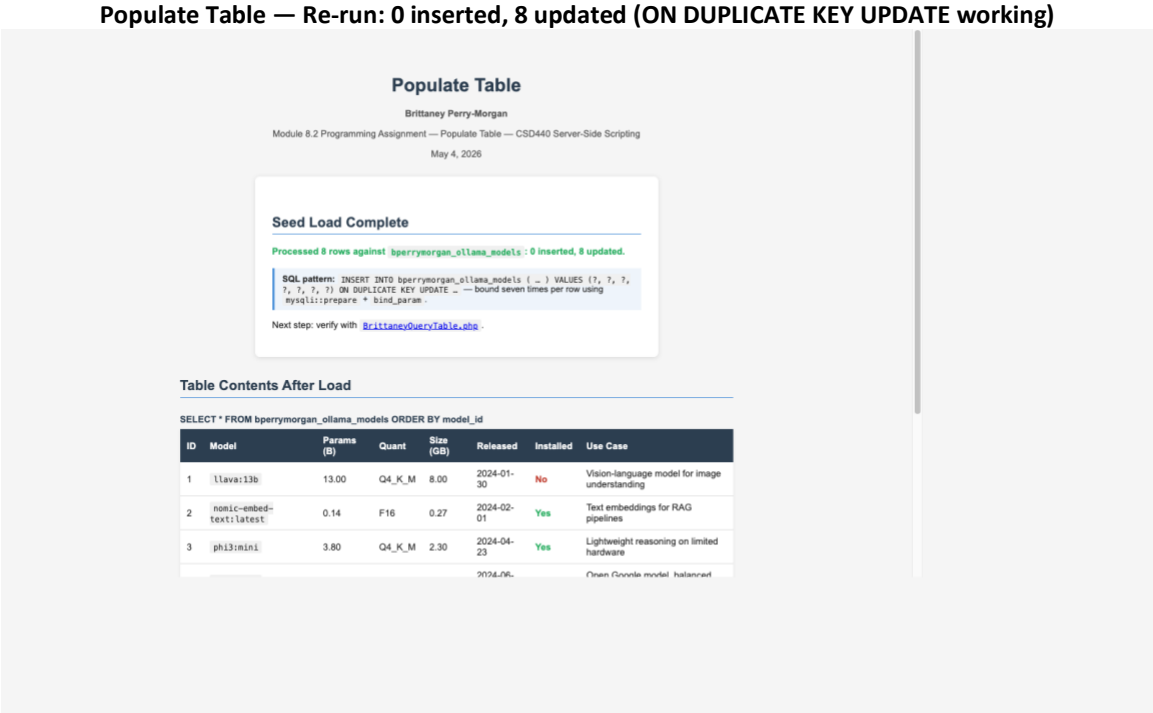
ID	Model	Params (B)	Quant	Size (GB)	Released	Installed	Use Case
1	`llava:13b`	13.00	Q4_K_M	8.00	2024-01-30	No	Vision-language model for image understanding
2	`nomic-embed-text:latest`	0.14	F16	0.27	2024-02-01	Yes	Text embeddings for RAG pipelines
3	`phi3:mini`	3.80	Q4_K_M	2.30	2024-04-23	Yes	Lightweight reasoning on limited hardware
4	`gemma2:9b`	9.00	Q4_K_M	5.44	2024-06-27	Yes	Open Google model, balanced performance
5	`mistral-nemo:12b`	12.00	Q4_K_M	7.07	2024-07-18	Yes	Multilingual chat with long context window
6	`llama3.1:8b`	8.00	Q4_K_M	4.92	2024-07-23	Yes	General-purpose chat and coding assistance
7	`llama3.1:70b`	70.00	Q4_K_M	39.50	2024-07-23	No	High-quality reasoning, slower local inference
8	`qwen2.5-coder:7b`	7.00	Q4_K_M	4.36	2024-09-19	Yes	Code generation and refactoring

Query Table | Create Table | Drop Table

Populate Table — Footer navigation and remaining data rows



Populate Table — Re-run: 0 inserted, 8 updated (ON DUPLICATE KEY UPDATE working)



Query Table — Section 1: All Models (newest first), 8 rows returned

Query Table

Brittaney Perry-Morgan

Module 8.2 Programming Assignment — Query Table — CSD440 Server-Side Scripting

May 4, 2026

1. All Models (newest first)

SQL: `SELECT * FROM bperrymorgan_ollama_models ORDER BY release_date DESC, model_name`

All 8 models

ID	Model	Params (B)	Quant	Size (GB)	Released	Installed	Use Case
8	`qwen2.5-coder:7b`	7.00	Q4_K_M	4.36	2024-09-19	Yes	Code generation and refactoring
7	`llama3.1:70b`	70.00	Q4_K_M	39.50	2024-07-23	No	High-quality reasoning, slower local inference
6	`llama3.1:8b`	8.00	Q4_K_M	4.92	2024-07-23	Yes	General-purpose chat and coding assistance
5	`mistral-nemo:12b`	12.00	Q4_K_M	7.07	2024-07-18	Yes	Multilingual chat with long context window
4	`gemma2:9b`	9.00	Q4_K_M	5.44	2024-06-27	Yes	Open Google model, balanced performance
3	`phi3:mini`	3.80	Q4_K_M	2.30	2024-04-23	Yes	Lightweight reasoning on limited hardware
2	`nomic-embed-text:latest`	0.14	F16	0.27	2024-02-01	Yes	Text embeddings for RAG pipelines
1	`llava:13b`	13.00	Q4_K_M	8.00	2024-01-30	No	Vision-language model for image understanding

Query Table — Section 2: Locally Installed Models, Largest First (6 rows, WHERE is_installed=1)

2. Locally Installed Models, Largest First

SQL: `SELECT model_name, parameter_count, size_gb, use_case FROM bperrymorgan_ollama_models WHERE is_installed = 1 ORDER BY parameter_count DESC`

6 installed models

Model	Params (B)	Size (GB)	Use Case
`mistral-nemo:12b`	12.00	7.07	Multilingual chat with long context window
`gemma2:9b`	9.00	5.44	Open Google model, balanced performance
`llama3.1:8b`	8.00	4.92	General-purpose chat and coding assistance
`qwen2.5-coder:7b`	7.00	4.36	Code generation and refactoring
`phi3:mini`	3.80	2.30	Lightweight reasoning on limited hardware
`nomic-embed-text:latest`	0.14	0.27	Text embeddings for RAG pipelines

3. Fleet Summary (aggregates)

SQL: `SELECT COUNT(*), SUM(is_installed), ROUND(SUM(size_gb),2), ROUND(AVG(parameter_count),2), MAX(parameter_count), MIN(release_date), MAX(release_date) FROM bperrymorgan_ollama_models`

Aggregate Statistics

Total models cataloged	8
Installed locally	6
Combined disk footprint	71.86 GB

Query Table — Section 3: Fleet Summary aggregates (COUNT, SUM, AVG, MAX, MIN)

Average parameter count	15.37 B
Largest model	70.00 B parameters
Oldest release date	2024-01-30
Newest release date	2024-09-19

4. Models Grouped by Quantization

```
SQL: SELECT quantization, COUNT(*), ROUND(AVG(size_gb),2) FROM bperrymorgan_ollama_models GROUP BY quantization
ORDER BY model_count DESC
```

Counts per quantization scheme

Quantization	Model Count	Avg Size (GB)
Q4_K_M	7	10.23
F16	1	0.27

Create Table | Populate Table | Drop Table

Query Table — Section 4: Models Grouped by Quantization (GROUP BY, 2 groups)

Brittaney Perry-Morgan

Module 8.2 Programming Assignment — Query Table — CSD440 Server-Side Scripting

May 4, 2026

1. All Models (newest first)

```
SQL: SELECT * FROM bperrymorgan_ollama_models ORDER BY release_date DESC, model_name
```

Query failed: Table 'baseball_01.bperrymorgan_ollama_models' doesn't exist

2. Locally Installed Models, Largest First

```
SQL: SELECT model_name, parameter_count, size_gb, use_case FROM bperrymorgan_ollama_models WHERE is_installed =
1 ORDER BY parameter_count DESC
```

Query failed: Table 'baseball_01.bperrymorgan_ollama_models' doesn't exist

3. Fleet Summary (aggregates)

```
SQL: SELECT COUNT(*), SUM(is_installed), ROUND(SUM(size_gb),2), ROUND(AVG(parameter_count),2),
MAX(parameter_count), MIN(release_date), MAX(release_date) FROM bperrymorgan_ollama_models
```

Query failed: Table 'baseball_01.bperrymorgan_ollama_models' doesn't exist

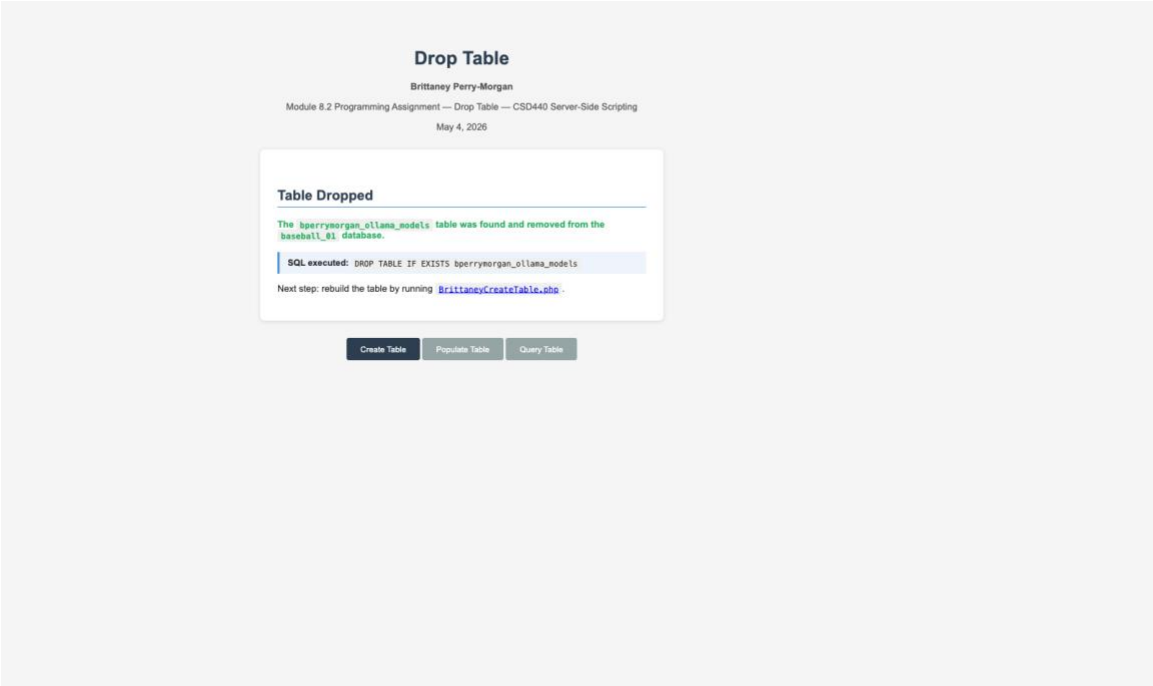
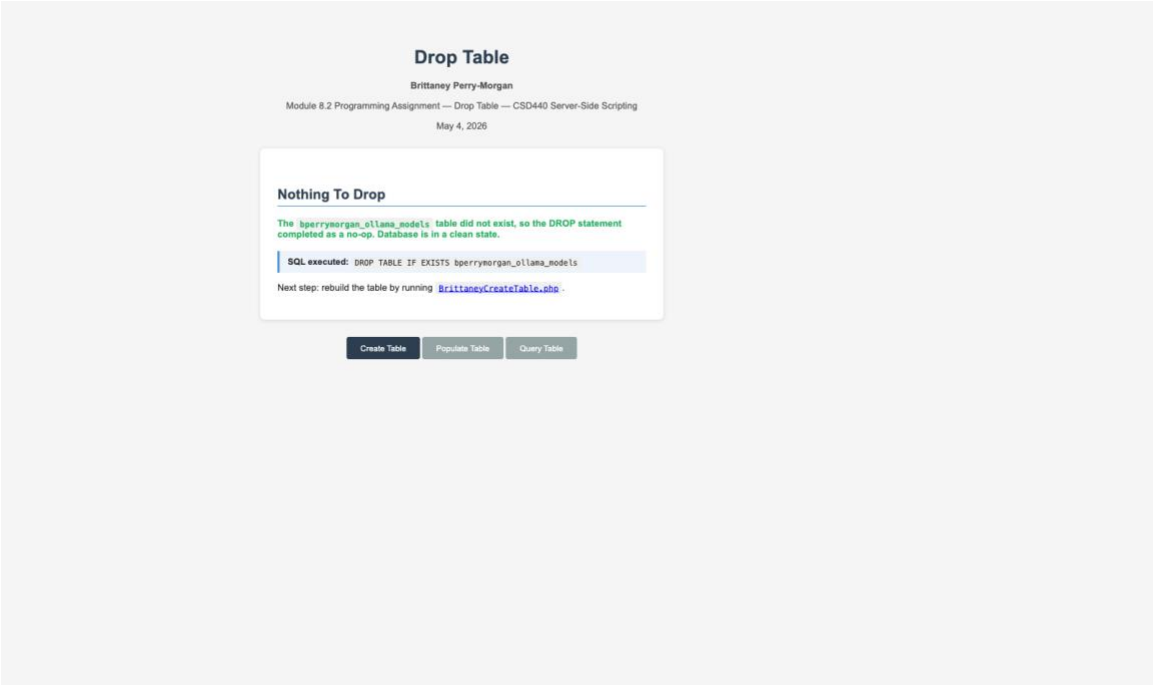
4. Models Grouped by Quantization

```
SQL: SELECT quantization, COUNT(*), ROUND(AVG(size_gb),2) FROM bperrymorgan_ollama_models GROUP BY quantization
ORDER BY model_count DESC
```

Query failed: Table 'baseball_01.bperrymorgan_ollama_models' doesn't exist

Create Table | Populate Table | Drop Table

Drop Table — Table found and removed (DROP TABLE IF EXISTS with data present)



Drop Table (idempotency) — Re-run: 'Nothing To Drop' (table already removed)